Day 18: Saga Pattern, Compensating Transactions & Orchestration

Coordinating long-running distributed workflows with explicit forward steps and rollback actions.

1 Why Saga Exists

- Split one business flow into local transactions
- Avoid strict global ACID transactions across services
- On failure, compensate completed steps in reverse order

2 Orchestration Model

- A central orchestrator decides the next step
- Each service exposes execute and compensate behavior
- Gives visibility, control, and deterministic rollback order

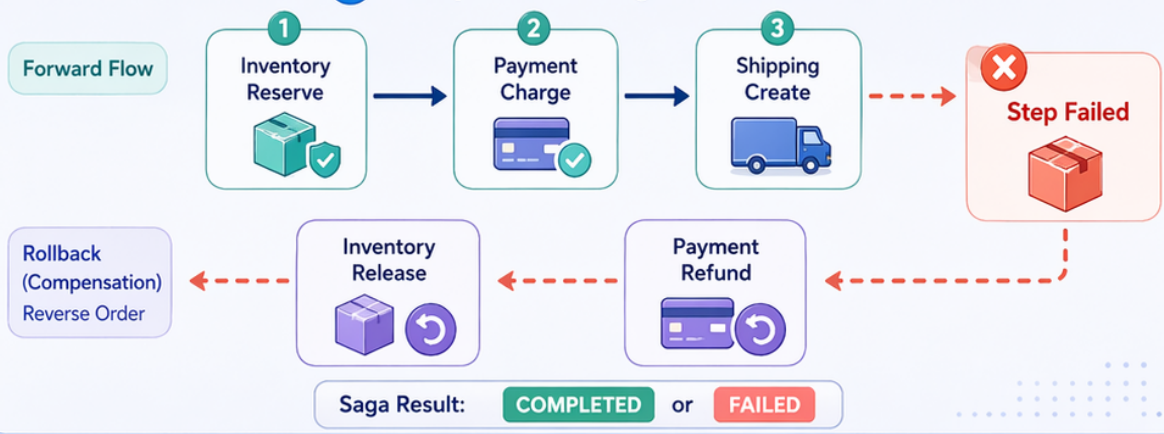
3 Compensation Behavior

- Compensation is a semantic undo, not a DB rollback
- Example: reserve inventory → release inventory
- Compensations should be idempotent

4 Failure Handling

- Failed step marks the saga as FAILED
- Orchestrator compensates completed steps in reverse order
- Retry, alerting, or manual recovery may be needed

5 Sample Order Saga Orchestrator



6 Trade-Offs

- Better availability than 2PC
- Eventual consistency instead of immediate global consistency
- Correctness depends on well-designed compensations

7 Key Takeaways

- Saga is practical for microservices workflows
- Idempotency is critical for execute and compensate paths
- Orchestration improves traceability for critical flows

Saga helps distributed systems stay consistent by coordinating local transactions with compensating actions.

System Design - Day 18

Saga Pattern, Compensating Transactions, and Orchestration

Prepared from Day-18 notes and sample implementation

Overview from README.md

Day 18: Saga Pattern, Compensating Transactions, and Orchestration

Day 18 focuses on how distributed systems keep business workflows consistent without using strict global transactions.

It covers:
- saga pattern fundamentals
- orchestration-based workflow control
- local transactions per service
- compensating actions for rollback behavior
- failure handling across multi-step flows
- eventual consistency trade-offs

In short, Day 18 is about coordinating long-running distributed operations with explicit forward and rollback steps.

Core ideas

Why Saga exists
- Cross-service workflows are hard to run under one ACID transaction.
- `Saga` splits a business flow into multiple local transactions.
- If one step fails, previously completed steps are compensated in reverse order.

Orchestration model
- A central orchestrator decides which step runs next.
- Each service exposes `execute` and compensation behavior.
- This model gives clear visibility and deterministic rollback order.

Compensation behavior
- Compensation is not a DB rollback; it is a semantic undo action.
- For example: reserve inventory -> later release inventory.
- Compensations must be idempotent because retries are common in distributed systems.

Failure handling
- A failed step marks the saga as failed.
- Orchestrator triggers compensations for already completed steps.
- If a compensation itself fails, the workflow enters a recovery/alert state and needs retries or manual intervention.

Trade-offs
- Saga improves availability and autonomy of services compared to 2PC.
- Consistency is eventual, not immediate across all services.
- Correctness depends heavily on well-designed compensating actions.

Day-18 sample: Order Saga Orchestrator

This repository includes a small Saga orchestration sample in both Python and Java.

Functional requirements
- Execute an order workflow across inventory, payment, and shipping steps

- Stop workflow when a step fails
- Trigger compensations for previously completed steps
- Record step-by-step execution history
- Surface final saga state (`COMPLETED` or `FAILED`)

High-level components
- `SagaStep`: interface/contract for execute and compensate
- `SagaOrchestrator`: coordinates step execution and rollback sequence
- Domain steps: inventory reserve/release, payment charge/refund, shipping create/cancel
- `SagaResult`: final outcome and execution log

Data flow
1. Orchestrator starts with ordered steps
2. Each step executes local business action
3. On success, step is tracked as completed
4. On failure, orchestrator compensates completed steps in reverse order
5. Saga result returns status and timeline for observability/debugging

Project structure

```
Day-18/
README.md
order-saga-sample/
python/
order_saga.py
demo.py
java/
OrderSaga.java
Main.java
```

Run the sample

Python

```
cd order-saga-sample\python
python demo.py
```

Java

```
cd order-saga-sample\java
javac Main.java OrderSaga.java
java Main
```

Interview takeaways
- Saga is a practical alternative to 2PC for high-availability microservices
- Compensation design is part of business modeling, not just engineering plumbing
- Orchestration gives stronger control/traceability than pure choreography for critical flows
- Idempotency is mandatory for both forward and compensating actions
- Eventual consistency should be explicit in user and API expectations

Next improvements
- Add retry/backoff policies for both execute and compensate paths
- Persist saga state so execution survives orchestrator restarts
- Add timeout handling and step-level circuit breakers
- Add dead-letter handling for failed compensations
- Add metrics and tracing IDs for each saga instance

Reference Implementation

Python and Java sample source files

File: order-saga-sample/python/demo.py

```
from order_saga import (
    InventoryStep,
    OrderSagaOrchestrator,
    PaymentStep,
    ShippingStep,
)


def print_result(title: str, orchestrator: OrderSagaOrchestrator) -> None:
    print(title)
    result = orchestrator.run()
    print("  status:", result.status.value)
    for line in result.history:
        print(" ", line)


def main() -> None:
    success_saga = OrderSagaOrchestrator(
        [InventoryStep(), PaymentStep(), ShippingStep()]
    )
    print_result("successful saga", success_saga)

    failed_saga = OrderSagaOrchestrator(
        [InventoryStep(), PaymentStep(), ShippingStep(should_fail=True)]
    )
    print("\n")
    print_result("failed saga with compensation", failed_saga)


if __name__ == "__main__":
    main()
```

File: order-saga-sample/python/order_saga.py

```python
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum


class SagaStatus(Enum):
    COMPLETED = "COMPLETED"
    FAILED = "FAILED"


@dataclass(frozen=True)
class SagaResult:
    status: SagaStatus
    history: list[str]


class SagaStep:
    def __init__(self, name: str) -> None:
        self.name = name

    def execute(self) -> bool:
        raise NotImplementedError

    def compensate(self) -> bool:
        raise NotImplementedError


class OrderSagaOrchestrator:
    def __init__(self, steps: list[SagaStep]) -> None:
        if not steps:
            raise ValueError("steps must be non-empty")
        self._steps = steps[:]

    def run(self) -> SagaResult:
        completed: list[SagaStep] = []
        history: list[str] = []

        for step in self._steps:
            history.append(f"execute {step.name}")
            ok = step.execute()
            history.append(f"result {step.name}: {'OK' if ok else 'FAIL'}")
            if not ok:
                history.append(f"saga failed at {step.name}; starting compensation")
                for completed_step in reversed(completed):
                    history.append(f"compensate {completed_step.name}")
                    compensated = completed_step.compensate()
                    history.append(
                        f"compensation {completed_step.name}: {'OK' if compensated else 'FAIL'}"
                    )
                return SagaResult(status=SagaStatus.FAILED, history=history)
            completed.append(step)

        history.append("saga completed")
        return SagaResult(status=SagaStatus.COMPLETED, history=history)


class InventoryStep(SagaStep):
    def __init__(self) -> None:
        super().__init__("inventory")
        self._reserved = False

    def execute(self) -> bool:
        self._reserved = True
        return True

    def compensate(self) -> bool:
        if self._reserved:
            self._reserved = False
        return True
class PaymentStep(SagaStep):
    def __init__(self, should_fail: bool = False) -> None:
```

```
        super().__init__("payment")
        self._should_fail = should_fail
        self._charged = False

    def execute(self) -> bool:
        if self._should_fail:
            return False
        self._charged = True
        return True

    def compensate(self) -> bool:
        if self._charged:
            self._charged = False
        return True


class ShippingStep(SagaStep):
    def __init__(self, should_fail: bool = False) -> None:
        super().__init__("shipping")
        self._should_fail = should_fail
        self._created = False

    def execute(self) -> bool:
        if self._should_fail:
            return False
        self._created = True
        return True

    def compensate(self) -> bool:
        if self._created:
            self._created = False
        return True
```

File: order-saga-sample/java/Main.java

```
import java.util.List;

public class Main {
    public static void main(String[] args) {
        OrderSaga.OrderSagaOrchestrator successSaga = new OrderSaga.OrderSagaOrchestrator(
            List.of(
                new OrderSaga.InventoryStep(),
                new OrderSaga.PaymentStep(false),
                new OrderSaga.ShippingStep(false)
            )
        );
        printResult("successful saga", successSaga);

        OrderSaga.OrderSagaOrchestrator failedSaga = new OrderSaga.OrderSagaOrchestrator(
            List.of(
                new OrderSaga.InventoryStep(),
                new OrderSaga.PaymentStep(false),
                new OrderSaga.ShippingStep(true)
            )
        );
        System.out.println();
        printResult("failed saga with compensation", failedSaga);
    }

    private static void printResult(String title, OrderSaga.OrderSagaOrchestrator orchestrator) {
        System.out.println(title);
        OrderSaga.SagaResult result = orchestrator.run();
        System.out.println("  status: " + result.getStatus());
        for (String line : result.getHistory()) {
            System.out.println("  " + line);
        }
    }
}
```

File: order-saga-sample/java/OrderSaga.java

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class OrderSaga {
    public enum SagaStatus {
        COMPLETED,
        FAILED
    }

    public static final class SagaResult {
        private final SagaStatus status;
        private final List<String> history;

        private SagaResult(SagaStatus status, List<String> history) {
            this.status = status;
            this.history = history;
        }

        public SagaStatus getStatus() {
            return status;
        }

        public List<String> getHistory() {
            return history;
        }
    }

    public interface SagaStep {
        String name();
        boolean execute();
        boolean compensate();
    }

    public static final class OrderSagaOrchestrator {
        private final List<SagaStep> steps;

        public OrderSagaOrchestrator(List<SagaStep> steps) {
            if (steps == null || steps.isEmpty()) {
                throw new IllegalArgumentException("steps must be non-empty");
            }
            this.steps = new ArrayList<>(steps);
        }

        public SagaResult run() {
            List<SagaStep> completed = new ArrayList<>();
            List<String> history = new ArrayList<>();

            for (SagaStep step : steps) {
                history.add("execute " + step.name());
                boolean ok = step.execute();
                history.add("result " + step.name() + ": " + (ok ? "OK" : "FAIL"));
                if (!ok) {
                    history.add("saga failed at " + step.name() + "; starting compensation");
                    Collections.reverse(completed);
                    for (SagaStep completedStep : completed) {
                        history.add("compensate " + completedStep.name());
                        boolean compensated = completedStep.compensate();
                        history.add(
                            "compensation "
                                + completedStep.name()
                                + ": "
                                + (compensated ? "OK" : "FAIL")
                        );
                    }
                    return new SagaResult(SagaStatus.FAILED, history);
                }
                completed.add(step);
            }

            history.add("saga completed");
            return new SagaResult(SagaStatus.COMPLETED, history);
```

```
    }
}

public static final class InventoryStep implements SagaStep {
    private boolean reserved = false;

    @Override
    public String name() {
        return "inventory";
    }

    @Override
    public boolean execute() {
        reserved = true;
        return true;
    }

    @Override
    public boolean compensate() {
        if (reserved) {
            reserved = false;
        }
        return true;
    }
}

public static final class PaymentStep implements SagaStep {
    private final boolean shouldFail;
    private boolean charged = false;

    public PaymentStep(boolean shouldFail) {
        this.shouldFail = shouldFail;
    }

    @Override
    public String name() {
        return "payment";
    }

    @Override
    public boolean execute() {
        if (shouldFail) {
            return false;
        }
        charged = true;
        return true;
    }

    @Override
    public boolean compensate() {
        if (charged) {
            charged = false;
        }
        return true;
    }
}

public static final class ShippingStep implements SagaStep {
    private final boolean shouldFail;
    private boolean created = false;

    public ShippingStep(boolean shouldFail) {
        this.shouldFail = shouldFail;
    }

    @Override
    public String name() {
        return "shipping";
    }

    @Override
    public boolean execute() {
        if (shouldFail) {
```

```
                return false;
            }
            created = true;
            return true;
        }

        @Override
        public boolean compensate() {
            if (created) {
                created = false;
            }
            return true;
        }
    }
}
```